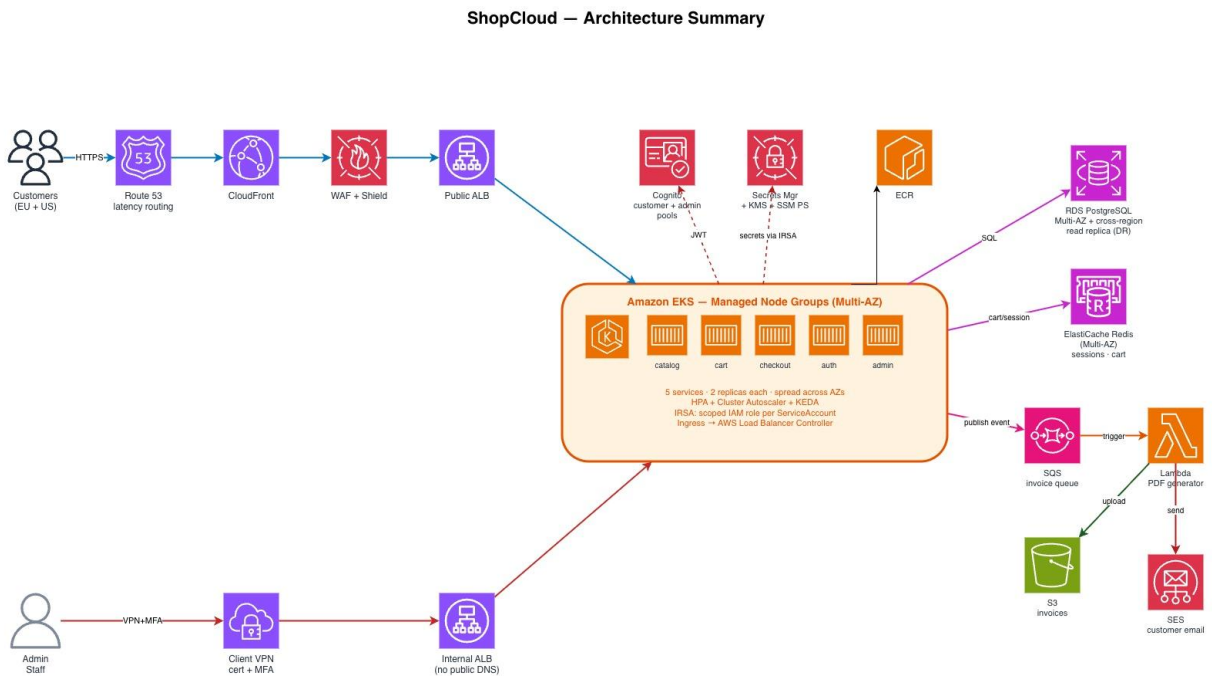
Project Architecture



Below is a description of the ShopCloud project's AWS infrastructure. This represents a **basic version**, and you are encouraged to enhance it by adding any elements you consider necessary. Please ensure that your design takes into account networking, security, and reliability.

Customer path (public, internet-facing)

Customers reach the application over HTTPS. **Route 53** performs latency-based routing to send each user to the nearest healthy endpoint. Traffic then flows through **CloudFront** (CDN for caching and TLS termination at the edge), protected by **AWS WAF + Shield** against common web attacks and DDoS. Requests are then forwarded to a **Public Application Load Balancer**, which routes them into the EKS cluster via the AWS Load Balancer Controller.

Admin path (private, restricted)

Administrative staff never touch the public front door. They connect through **AWS Client VPN**, authenticated with a certificate *and* MFA, which gives them access to an **Internal ALB** that has no public DNS record. This internal ALB forwards admin traffic into the same EKS cluster but through a separate ingress.

The EKS workload layer

Amazon EKS cluster hosts five microservices — `catalog`, `cart`, `checkout`, `auth`, and `admin` — each running with at least 2 replicas distributed across AZs for high availability.

Identity, secrets, and images

- **Amazon Cognito** handles authentication with two separate user pools (customers and admins) and issues **JWTs** consumed by the services.
- **Secrets Manager**, **KMS**, and **SSM Parameter Store** store credentials and config, retrieved by pods through their IRSA roles — no secrets are baked into images or env files.
- **Amazon ECR** hosts the container images pulled by EKS at deploy time.

Data layer

- **Amazon RDS for PostgreSQL** is the primary transactional store, deployed **Multi-AZ** for HA and with a **cross-region read replica** for disaster recovery.
- **Amazon ElastiCache for Redis** (Multi-AZ) handles low-latency session state and shopping carts.

Asynchronous invoicing pipeline

When a checkout completes, the `checkout` service **publishes an event to Amazon SQS** (the invoice queue). This decouples the user-facing request from heavier downstream work. The queue **triggers an AWS Lambda function** (the PDF generator), which renders the invoice, **uploads it to an S3 bucket**, and then uses **Amazon SES** to email the PDF to the customer. KEDA can scale consumers based on SQS depth if the event stream gets bursty.

Bonus

Making the application available for customers in EU and US with low latency.